**UNIVERSITÉ MOHAMED V**

École Nationale Supérieure d'Arts et Métiers

Rabat - Maroc


RAPPORT TECHNIQUE

# TP3

## PROJET ML COMPLET

### Analyse de Sentiment YouTube

*Du Développement au Déploiement Cloud*

**Python** **FastAPI** **Cloud** **ML**

## INFORMATIONS DU PROJET

**Module :**
Virtualisation & Cloud Computing

**Filière :**
INDIA

**Professeur :**
Prof. Maria Zemzami

**RÉALISÉ PAR**

**LAKHAL Hala**

**Année Universitaire**
2025/2026

**Date de Remise : 26 Novembre 2025**

# Table des matières


**1 Introduction** **3**
1.1 Contexte du Projet . . . 3
1.2 Objectifs du Projet . . . 3
1.3 Architecture Globale . . . 3

**2 Phase 1 : Configuration de l'Environnement MLOps** **4**
2.1 Structure du Projet . . . 4
2.2 Questions de Réflexion - Réponses . . . 4

**3 Phase 2 : Pipeline de Données et Préparation** **5**
3.1 Dataset Utilisé . . . 5
3.2 Nettoyage des Données . . . 5
3.3 Analyse Exploratoire . . . 5
3.4 Questions de Réflexion - Réponses . . . 6

**4 Phase 3 : Développement et Entraînement du Modèle** **7**
4.1 Architecture du Modèle . . . 8
4.2 Paramètres du Vectoriseur TF-IDF . . . 8
4.3 Optimisation des Hyperparamètres . . . 8
4.4 Matrice de Confusion . . . 9
4.5 Rapport de Classification Détaillé . . . 10
4.6 Questions de Réflexion - Réponses . . . 10

**5 Phase 4 : Développement de l'API REST** **11**
5.1 Architecture de l'API . . . 11
5.2 Endpoints Implémentés . . . 11
5.3 Exemple de Requête et Réponse . . . 11
5.4 Tests de l'API . . . 13
5.5 Questions de Réflexion - Réponses . . . 14

**6 Phase 5 : Développement de l'Extension Chrome** **15**
6.1 Architecture de l'Extension . . . 15
6.2 Fonctionnalités Implémentées . . . 15
6.3 Questions de Réflexion - Réponses . . . 16

**7 Phase 6 : Déploiement Cloud avec Docker** **16**
7.1 Configuration Docker . . . 16
7.2 Déploiement sur Hugging Face Spaces . . . 17
7.3 Tests de l'API Déployée . . . 17
7.4 Questions de Réflexion - Réponses . . . 18

**8 Phase 7 : Tests et Validation** **19**
8.1 Tests du Modèle . . . 19
8.1.1 Performance sur le Test Set . . . 19
8.1.2 Tests de Cas Limites . . . 20
8.1.3 Tests de Performance . . . 20
8.2 Tests de l'Extension Chrome . . . 20

8.2.1 Installation et Chargement . . . . . 20
8.2.2 Test sur une Vidéo YouTube Réelle . . . . . 21
8.2.3 Extension . . . . . 21
8.3 Tests d'Intégration . . . . . 21

**9 Résultats et Discussion** **22**
9.1 Analyse des Résultats . . . . . 22
9.1.1 Points Forts . . . . . 22
9.1.2 Limitations Identifiées . . . . . 22
9.2 Difficultés Rencontrées et Solutions . . . . . 23

**10 Améliorations Futures Possibles** **23**
10.1 Améliorations du Modèle . . . . . 23
10.2 Améliorations de l'Extension . . . . . 23
10.3 Améliorations de l'Infrastructure . . . . . 23

**11 Conclusion** **24**
11.1 Synthèse du Projet . . . . . 24
11.2 Compétences Acquises . . . . . 24
11.3 Perspectives . . . . . 24

**12 Annexes** **25**
12.1 Annexe A : Liens et Ressources . . . . . 25
12.2 Annexe B : Structure Complète du Code . . . . . 25
12.3 Annexe C : Commandes Principales . . . . . 26

**13 Références** **26**

# 1 Introduction

## 1.1 Contexte du Projet

Dans l'écosystème actuel de YouTube, les créateurs de contenu font face à un défi majeur : l'analyse des milliers de commentaires qu'ils reçoivent. Cette tâche, essentielle pour comprendre l'opinion de leur audience, est :

— **Chronophage :** Impossibilité de lire manuellement des milliers de commentaires

— **Subjective :** L'analyse humaine peut être biaisée

— **Non scalable :** Difficile à quantifier pour des décisions stratégiques

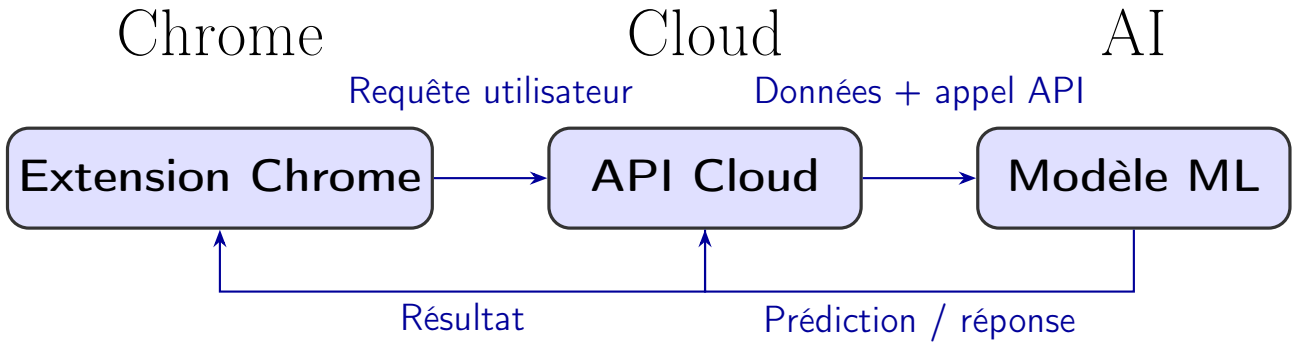
## 1.2 Objectifs du Projet

Ce projet vise à développer un système MLOps complet permettant d'analyser automatiquement le sentiment des commentaires YouTube en temps réel. Les objectifs spécifiques sont :

1. Développer un modèle de Machine Learning performant (Accuracy $\geq 80\%$)
2. Créer une API REST déployée dans le cloud
3. Développer une extension Chrome pour l'interface utilisateur
4. Mettre en place un pipeline de tests et validation

## 1.3 Architecture Globale

Le système implémente une architecture complète illustrée dans la Figure ci-dessous.



# 2 Phase 1 : Configuration de l'Environnement MLOps

## 2.1 Structure du Projet

Nous avons mis en place une structure de projet MLOps suivant les bonnes pratiques de l'industrie :

Listing 1 – Structure des dossiers

```
youtube-sentiment-analysis/
          data/
                raw/                # Donnees brutes
                processed/          # Donnees nettoyees
          models/                 # Modeles entraines (.joblib)
          src/
                data/             # Scripts de traitement
                models/           # Scripts d'entrainement
                api/              # Code de l'API
                utils/            # Fonctions utilitaires
          chrome-extension/     # Extension Chrome
                js/
                css/
                icons/
          tests/                # Tests unitaires
          logs/                 # Fichiers de logs
          requirements.txt      # Dependances Python
          Dockerfile           # Configuration Docker
          README.md            # Documentation
```

## 2.2 Questions de Réflexion - Réponses

### Pourquoi séparer les données raw et processed ?

La séparation entre données brutes (`raw`) et traitées (`processed`) est cruciale pour :

- **Reproductibilité :** Possibilité de retraiter les données avec différents paramètres
- **Traçabilité :** Conservation de l'état original des données
- **Debugging :** Facilite l'identification des problèmes de prétraitement
- **Versionning :** Permet de tester différentes stratégies de nettoyage

### Quels fichiers ne doivent jamais être commités sur Git ?

Les fichiers à exclure du versioning Git incluent :

- **Données volumineuses :** `*.csv`, datasets (utiliser Git LFS si nécessaire)
- **Modèles entraînés :** `*.joblib`, `*.pkl` (trop volumineux)
- **Environnements virtuels :** `venv/`, `env/`
- **Fichiers temporaires :** `__pycache__/`, `*.pyc`
- **Secrets :** `.env`, clés API, tokens
- **Logs :** `*.log`

**Quelle est l'importance d'un environnement virtuel ?**

Un environnement virtuel Python est essentiel pour :

— **Isolation :** Éviter les conflits entre dépendances de différents projets

— **Reproductibilité :** Garantir que le code fonctionne avec des versions spécifiques

— **Portabilité :** Faciliter le déploiement sur différentes machines

— **Gestion des dépendances :** Installer uniquement les packages nécessaires

# 3 Phase 2 : Pipeline de Données et Préparation

## 3.1 Dataset Utilisé

Nous avons utilisé le dataset **Reddit Sentiment Analysis** contenant des commentaires labelisés avec trois catégories :

— **-1 :** Sentiment Négatif

— **0 :** Sentiment Neutre

— **1 :** Sentiment Positif

## 3.2 Nettoyage des Données

Le processus de nettoyage comprend les étapes suivantes :

1. **Suppression des URLs :** Retrait des liens hypertextes
2. **Suppression des mentions :** Retrait des @username
3. **Nettoyage des hashtags :** Conservation du texte, suppression du #
4. **Suppression des caractères spéciaux :** Conservation uniquement des lettres et chiffres
5. **Normalisation :** Conversion en minuscules
6. **Suppression des espaces multiples**

## 3.3 Analyse Exploratoire

Après nettoyage, l'analyse exploratoire révèle :

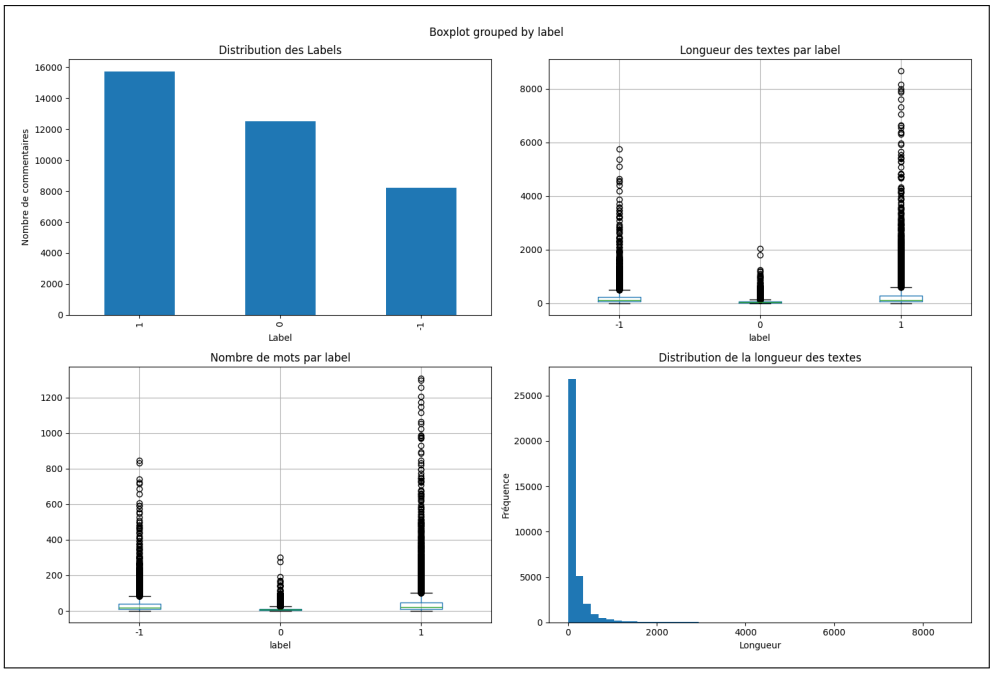



FIGURE 1 – Résultats du nettoyage des données

## 3.4 Questions de Réflexion - Réponses

### Pourquoi nettoyer les URLs et mentions ?

Le nettoyage des URLs et mentions est nécessaire car :

— **URLs :** Ne portent pas de sentiment intrinsèque et ajoutent du bruit

— **Mentions :** Sont spécifiques au contexte et ne généralisent pas

— **Vocabulaire :** Réduisent la taille du vocabulaire inutilement

— **Performance :** Améliorent la performance du modèle en réduisant les features non pertinentes

### Comment gérer les emojis dans le texte ?

Pour les emojis, plusieurs approches sont possibles :

— **Conservation :** Si le modèle peut les traiter (utile car porteurs de sentiment)

— **Conversion en texte :** Remplacer  par "happy",  par "sad"

— **Suppression :** Dans notre cas, supprimés avec les caractères spéciaux

Notre choix : suppression pour simplifier, mais la conservation avec conversion serait optimale.

### Quel ratio train/test est approprié ?

Nous avons choisi un ratio **80/20** (train/test) car :

— **Standard :** Ratio couramment utilisé en ML

— **Équilibré :** Suffisamment de données pour l'entraînement et la validation

— **Split stratifié :** Maintient la distribution des classes dans train et test

Pour des datasets plus petits, un ratio 70/30 ou une validation croisée seraient préférables.

# 4 Phase 3 : Développement et Entraînement du Modèle

```
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
$ python src/models/train_model.py
📂 Chargement des données...
✅ Données préparées:
   Train: 29160 samples
   Test: 7290 samples

🔤 Entraînement du vectoriseur TF-IDF...
✅ Vocabulaire: 5000 mots
✅ Shape des features: (29160, 5000)

🎯 Entraînement du modèle...
   Optimisation des hyperparamètres avec GridSearch...
Fitting 5 folds for each of 3 candidates, totalling 15 fits

✅ Meilleurs paramètres: {'C': 10, 'max_iter': 1000, 'penalty': 'l
1
bfgs'}
✅ Meilleur score F1: 0.8649
✅ Modèle entraîné!
```

FIGURE 2 – Développement et Entraînement du Modèle

```
✅ Modèle sauvegardé dans models/

============================================================
✅ ENTRAÎNEMENT TERMINÉ AVEC SUCCÈS!
============================================================
(venv)
```

FIGURE 3 – Développement et Entraînement du Modèle

```
📊 ÉVALUATION DU MODÈLE
============================================================

🎯 Accuracy: 0.8794
🎯 F1-Score (macro): 0.8660

📋 Rapport de classification:
              precision    recall  f1-score   support

     Négatif       0.83      0.74      0.78      1647
      Neutre       0.89      0.95      0.92      2499
     Positif       0.89      0.89      0.89      3144

    accuracy                           0.88      7290
   macro avg       0.87      0.86      0.87      7290
weighted avg       0.88      0.88      0.88      7290


✅ Matrice de confusion sauvegardée dans logs/confusion_matrix.png

⏱️Test de performance:
   Temps d'inférence pour 50 commentaires: 0.00ms
   Temps moyen par commentaire: 0.00ms
```

FIGURE 4 – Evaluation du modele

## 4.1 Architecture du Modèle

Le modèle implémenté combine deux composants :

1. **TF-IDF Vectorizer :** Transformation du texte en vecteurs numériques
2. **Logistic Regression :** Classification multi-classes

## 4.2 Paramètres du Vectoriseur TF-IDF

Listing 2 – Configuration TF-IDF

```python
vectorizer = TfidfVectorizer(
    max_features=5000,       # Vocabulaire limite
    ngram_range=(1, 2),      # Unigrams et bigrams
    min_df=2,                # Frequence min
    max_df=0.95,             # Frequence max
    sublinear_tf=True        # Echelle logarithmique
)
```

## 4.3 Optimisation des Hyperparamètres

Nous avons utilisé `GridSearchCV` pour optimiser les hyperparamètres :

| Hyperparamètre | Valeurs testées |
|---|---|
| C (régularisation) | [0.1, 1, 10] |
| Penalty | ['l2'] |
| Solver | ['lbfgs'] |
| Max iterations | [1000] |

TABLE 1 – Grille de recherche des hyperparamètres

**Meilleurs paramètres trouvés :**

— Penalty = l2

— Solver = lbfgs

## 4.4 Matrice de Confusion

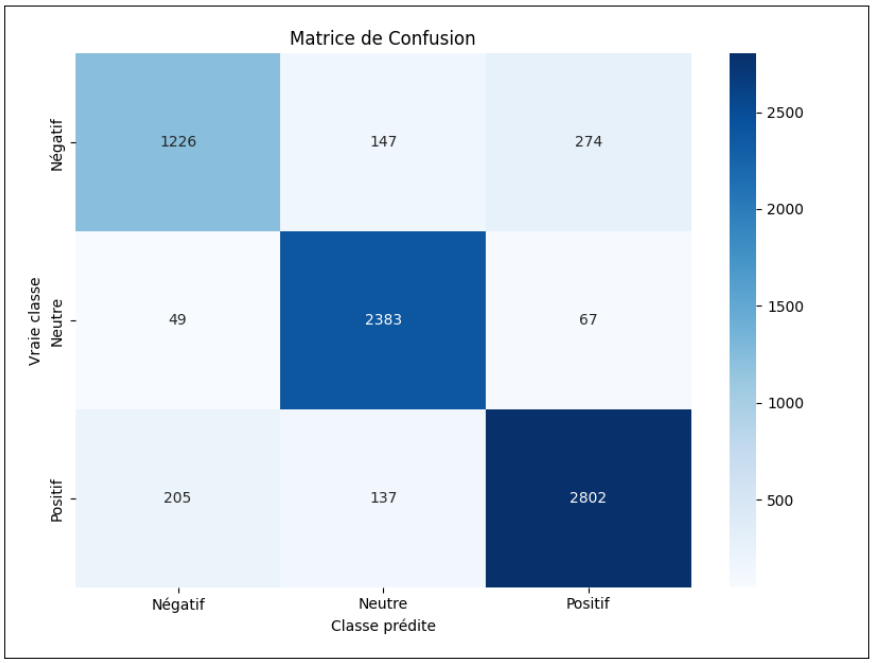



FIGURE 5 – Matrice de confusion du modèle

## 4.5 Rapport de Classification Détaillé

```
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
$ python src/models/train_model.py
📂 Chargement des données...
✅ Données préparées:
   Train: 29160 samples
   Test: 7290 samples

🔤 Entraînement du vectoriseur TF-IDF...
✅ Vocabulaire: 5000 mots
✅ Shape des features: (29160, 5000)

🎯 Entraînement du modèle...
   Optimisation des hyperparamètres avec GridSearch...
Fitting 5 folds for each of 3 candidates, totalling 15 fits

✅ Meilleurs paramètres: {'C': 10, 'max_iter': 1000, 'penalty': 'l
1
bfgs'}
✅ Meilleur score F1: 0.8649
✅ Modèle entraîné!
```

FIGURE 6 – Rapport de classification complet

```
✅ Modèle sauvegardé dans models/

============================================================
✅ ENTRAÎNEMENT TERMINÉ AVEC SUCCÈS!
============================================================
(venv)
```

FIGURE 7 – Rapport de classification complet

## 4.6 Questions de Réflexion - Réponses

**Pourquoi utiliser TF-IDF plutôt que Count Vectorizer ?**

TF-IDF (Term Frequency-Inverse Document Frequency) est préférable car :

- **Pénalisation des mots fréquents :** Réduit l'importance des mots comme "le", "la", "et"
- **Valorisation des mots rares :** Met en avant les termes discriminants
- **Meilleure représentation :** Capture mieux l'importance relative des mots
- **Performance supérieure :** Généralement meilleures performances en classification

Count Vectorizer ne fait que compter les occurrences sans pondération.

**Comment interpréter la matrice de confusion ?**

La matrice de confusion se lit ainsi :

- **Diagonale :** Prédictions correctes (VP pour chaque classe)
- **Hors diagonale :** Erreurs de classification
- **Lignes :** Classes réelles
- **Colonnes :** Classes prédites

Par exemple, si confusion[0][1] = 10 : 10 commentaires négatifs ont été classés comme neutres.

**Quels hyperparamètres ont le plus d'impact ?**

Les hyperparamètres les plus impactants sont :

1. **C (régularisation) :** Contrôle l'overfitting
   - C élevé : moins de régularisation, risque d'overfitting
   - C faible : plus de régularisation, risque d'underfitting
2. **max_features (TF-IDF) :** Taille du vocabulaire
   - Impact direct sur la complexité et la performance
   - Compromis entre richesse et bruit
3. **ngram_range :** Capture de contexte
   - (1,1) : mots seuls
   - (1,2) : mots + paires de mots (meilleur contexte)

# 5 Phase 4 : Développement de l'API REST

## 5.1 Architecture de l'API

L'API est développée avec **FastAPI**, un framework moderne et performant pour Python.

## 5.2 Endpoints Implémentés

| Endpoint | Méthode | Description |
|---|---|---|
| `/` | GET | Page d'accueil avec informations sur l'API |
| `/health` | GET | Vérification de l'état de l'API et des modèles |
| `/predict_batch` | POST | Analyse de sentiment pour un batch de commentaires |

Table 2 – Endpoints de l'API

## 5.3 Exemple de Requête et Réponse

**Requête POST vers /predict_batch :**

Listing 3 – Exemple de requête

```
{
  "comments": [
    "This video is amazing!",
    "I don't like this content",
    "It's okay, nothing special"
  ]
}
```

**Réponse :**

Listing 4 – Exemple de réponse

```
{
  "results": [
    {
      "text": "This video is amazing!",
      "sentiment": 1,
      "sentiment_label": "Positif",
      "confidence": 0.923
    },
    {
      "text": "I don't like this content",
      "sentiment": -1,
      "sentiment_label": "Negatif",
      "confidence": 0.887
    },
    {
      "text": "It's okay, nothing special",
      "sentiment": 0,
      "sentiment_label": "Neutre",
      "confidence": 0.756
    }
  ],
  "statistics": {
    "total_comments": 3,
    "positive": 1,
    "neutral": 1,
    "negative": 1,
    "positive_percentage": 33.33,
    "neutral_percentage": 33.33,
    "negative_percentage": 33.33,
    "average_confidence": 0.855
  }
}
```

## 5.4 Tests de l'API

```
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
is (master)
$ curl http://localhost:7860/health
{"status":"healthy","model_loaded":true,"vectorizer_loaded":true}(
venv)
```

FIGURE 8 – Test du endpoint /health

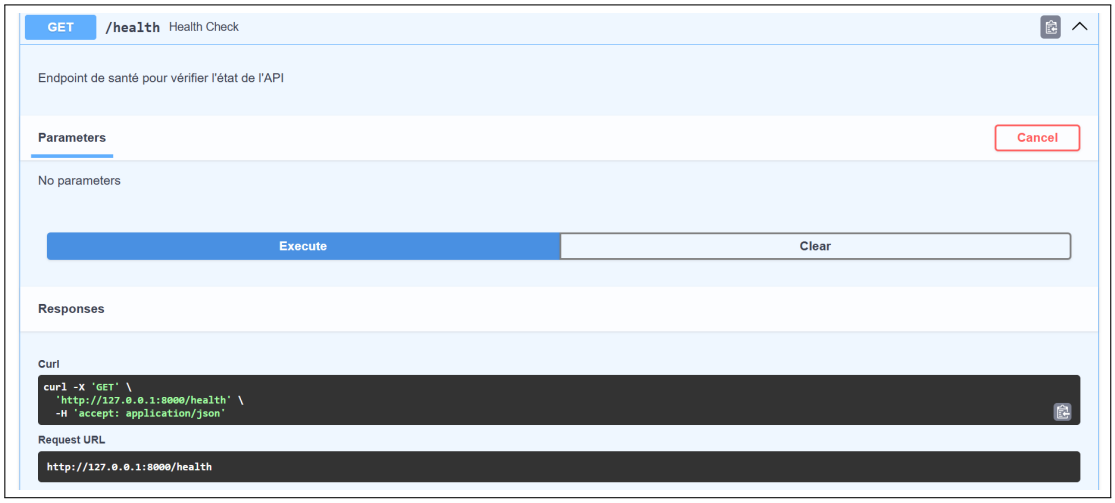



FIGURE 9 – Test du endpoint /health

FIGURE 10 – Test du endpoint /health

FIGURE 11 – Test du endpoint /predict_batch

FIGURE 12 – Test du endpoint /predict_batch

## 5.5 Questions de Réflexion - Réponses

### Pourquoi utiliser FastAPI plutôt que Flask ?

FastAPI présente plusieurs avantages sur Flask :

- **Performance :** Basé sur Starlette et Pydantic, très rapide (comparable à Node.js)
- **Validation automatique :** Validation des données avec Pydantic
- **Documentation auto :** Génération automatique de Swagger UI
- **Type hints :** Support natif des type hints Python
- **Async :** Support natif de l'asynchrone
- **Moderne :** Conçu pour les API REST modernes

### À quoi sert la configuration CORS ?

CORS (Cross-Origin Resource Sharing) permet :

- **Accès cross-domain :** L'extension Chrome peut appeler l'API depuis un autre domaine

— **Sécurité :** Contrôle quels domaines peuvent accéder à l'API
— **Headers personnalisés :** Autorisation des headers HTTP nécessaires
— **Méthodes HTTP :** Spécification des méthodes autorisées (GET, POST, etc.)

Sans CORS, le navigateur bloquerait les requêtes de l'extension vers l'API.

**Comment gérer efficacement le chargement du modèle ?**

Stratégies pour un chargement efficace :

1. **Chargement au démarrage :** Utiliser `@app.on_event("startup")`
   — Évite de recharger le modèle à chaque requête
   — Temps de réponse constant et rapide
2. **Variable globale :** Stocker le modèle en mémoire
   — Accès instantané pour les prédictions
   — Réutilisation entre requêtes
3. **Gestion d'erreurs :** Vérifier que le modèle est chargé
   — Retourner 503 si le modèle n'est pas disponible
   — Logs pour le debugging

# 6 Phase 5 : Développement de l'Extension Chrome

## 6.1 Architecture de l'Extension

L'extension Chrome est composée de :

— **manifest.json :** Configuration et permissions
— **content.js :** Script d'extraction des commentaires YouTube
— **popup.html/css :** Interface utilisateur
— **popup.js :** Logique et communication avec l'API

## 6.2 Fonctionnalités Implémentées

1. **Extraction automatique** des commentaires YouTube
2. **Analyse en temps réel** via l'API cloud
3. **Statistiques globales** (pourcentages, répartition)
4. **Liste détaillée** des commentaires avec sentiment et confiance
5. **Filtres** par sentiment (Positif/Neutre/Négatif/Tous)
6. **Mode sombre/clair** avec sauvegarde de préférence
7. **Copie des résultats** dans le presse-papiers
8. **Export CSV** pour analyse ultérieure

## 6.3 Questions de Réflexion - Réponses

**Comment gérer les commentaires avec réponses (threads) ?**

Pour gérer les threads de commentaires :

- **Sélecteur DOM :** Utiliser des sélecteurs spécifiques pour les commentaires principaux
- **Hiérarchie :** Distinguer commentaires parents et réponses
- **Options :**
  - Analyser uniquement les commentaires principaux
  - Analyser tout (commentaires + réponses)
  - Laisser l'utilisateur choisir

Notre choix : extraction des commentaires principaux uniquement pour simplifier.

**Quelle stratégie pour extraire les commentaires dynamiques ?**

YouTube charge les commentaires de manière dynamique (infinite scroll) :

- **Extraction à la demande :** Extraire uniquement les commentaires visibles
- **Scroll automatique :** Implémenter un scroll pour charger plus de commentaires
- **Observer API :** Détecter l'ajout de nouveaux commentaires
- **Limitation :** Éviter de charger trop de commentaires (performance)

Notre implémentation : extraction des commentaires actuellement visibles dans le DOM.

**Comment optimiser les performances avec de nombreux commentaires ?**

Optimisations pour gérer de nombreux commentaires :

- **Traitement par batch :** Envoyer les commentaires par groupe
- **Pagination :** Afficher les résultats de manière paginée
- **Virtualisation :** Utiliser le virtual scrolling pour l'affichage
- **Limitation :** Fixer une limite maximale (ex : 1000 commentaires)
- **Indicateurs de chargement :** Afficher la progression

# 7 Phase 6 : Déploiement Cloud avec Docker

## 7.1 Configuration Docker

Le Dockerfile optimisé pour Hugging Face Spaces :

Listing 5 – Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements_prod.txt .
```

```
RUN pip install --no-cache-dir -r requirements_prod.txt

COPY app_api.py .
COPY models/sentiment_model.joblib ./sentiment_model.joblib
COPY models/vectorizer.joblib ./vectorizer.joblib

EXPOSE 7860

CMD ["uvicorn", "app_api:app", "--host", "0.0.0.0", "--port",
    "7860"]
```

## 7.2 Déploiement sur Hugging Face Spaces

**URL de l'API déployée :**

https://halalakhal-youtube-sentiment-api.hf.space

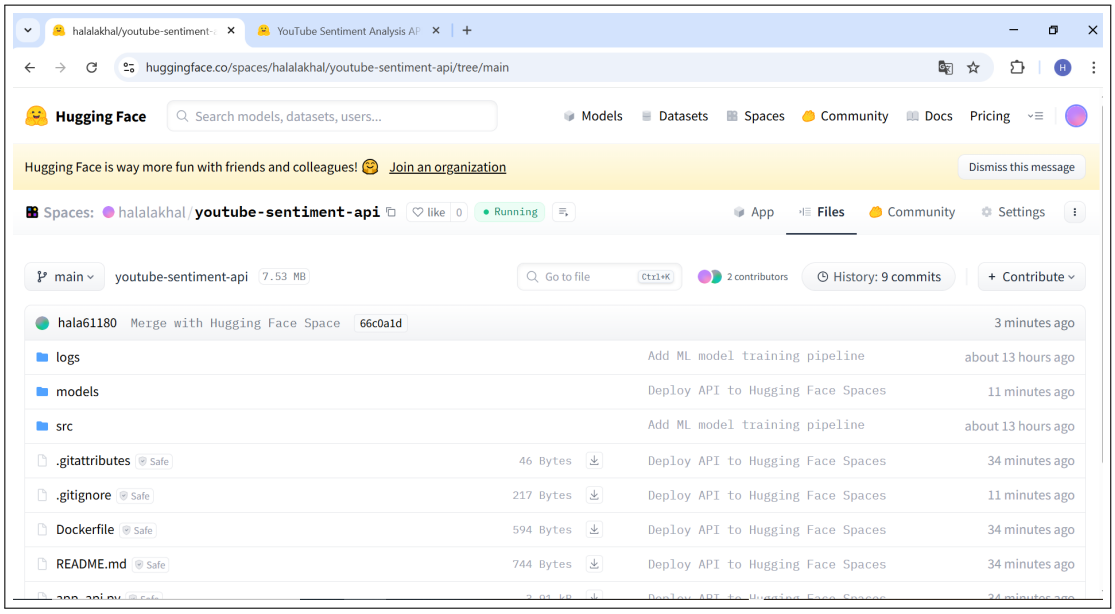



FIGURE 13 – Space Hugging Face en état "Running"

## 7.3 Tests de l'API Déployée

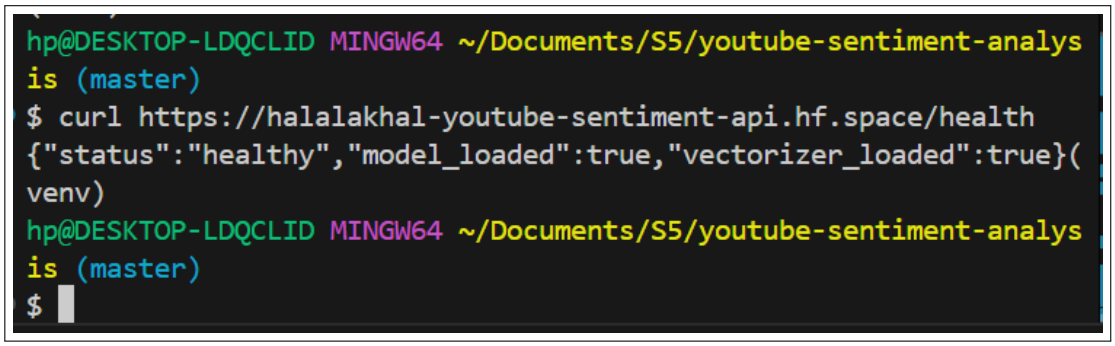



FIGURE 14 – Test de l'API déployée sur le cloud

```
venv)
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
is (master)
$ curl -X POST https://halalakhal-youtube-sentiment-api.hf.space/p
redict_batch \
>   -H "Content-Type: application/json" \
>   -d '{"comments": ["This video is amazing!", "I dont like it",
"Its okay"]}'
{"results":[{"text":"This video is amazing!","sentiment":1,"sentim
ent_label":"Positif","confidence":0.989},{"text":"I dont like it",
"sentiment":0,"sentiment_label":"Neutre","confidence":0.951},{"tex
t":"Its okay","sentiment":1,"sentiment_label":"Positif","confidenc
e":0.995}],"statistics":{"total_comments":3,"positive":2,"neutral"
:1,"negative":0,"positive_percentage":66.67,"neutral_percentage":3
3.33,"negative_percentage":0.0,"average_confidence":0.978}}(venv)
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
is (master)
$
```

FIGURE 15 – Test de l'API déployée sur le cloud

## 7.4 Questions de Réflexion - Réponses

### Pourquoi séparer les requirements dev et prod ?

Séparation nécessaire pour :

- **Taille de l'image :** requirements_prod contient uniquement les dépendances essentielles
  - Pas de matplotlib, seaborn (visualisation)
  - Pas de jupyter, pytest (développement)
  - Image Docker plus légère et rapide à déployer
- **Sécurité :** Moins de packages = moins de vulnérabilités potentielles
- **Performance :** Installation plus rapide au déploiement
- **Clarté :** Distingue clairement ce qui est nécessaire en production

### Comment réduire la taille de l'image Docker ?

Techniques d'optimisation :

1. **Image de base légère :** Utiliser `python:3.10-slim` au lieu de `python:3.10`
   - Économie de plusieurs centaines de Mo
2. **Multi-stage build :** Séparer la phase de build et d'exécution
3. **–no-cache-dir :** Éviter le cache pip
4. **Minimiser les layers :** Combiner les commandes RUN
5. **.dockerignore :** Exclure les fichiers inutiles
6. **Copie sélective :** Copier uniquement les fichiers nécessaires

**Quelle stratégie pour les mises à jour du modèle ?**

Stratégies de mise à jour :

- **CI/CD :** Pipeline automatique pour tester et déployer
- **Versioning :** Nommer les modèles avec version (model_v1.joblib, model_v2.joblib)
- **A/B Testing :** Déployer progressivement et comparer performances
- **Rollback :** Possibilité de revenir à la version précédente
- **Monitoring :** Suivre les performances en production

# 8 Phase 7 : Tests et Validation

## 8.1 Tests du Modèle

### 8.1.1 Performance sur le Test Set

```
hp@DESKTOP-LDQCLID MINGW64 ~/Documents/S5/youtube-sentiment-analys
is (master)
$ python tests/test_model.py
🧪 TEST 1: Performance sur le test set
==================================================
Accuracy: 0.9427

Rapport de classification:
              precision    recall  f1-score   support

     Négatif       0.93      0.88      0.90      8233
      Neutre       0.94      0.98      0.96     12496
     Positif       0.95      0.95      0.95     15721

    accuracy                           0.94     36450
   macro avg       0.94      0.93      0.94     36450
weighted avg       0.94      0.94      0.94     36450

✅ Test réussi: Accuracy >= 80%
```

FIGURE 16 – Résultats des tests du modèle

### 8.1.2 Tests de Cas Limites

```
🧪 TEST 2: Cas limites
==================================================
✓ Texte vide: 0
✓ Texte très court: 0
✓ Texte très long: -1
✓ Emojis seulement: 0
✓ Majuscules et ponctuation: 1
✓ Mélange positif/négatif: 1
```

FIGURE 17 – Résultats des tests du modèle

### 8.1.3 Tests de Performance

```
🧪 TEST 3: Temps d'inférence
==================================================
Temps pour 50 commentaires: 0.00ms
✅ Test réussi: Temps d'inférence < 100ms

==================================================
🎉 Tous les tests sont terminés!
```

FIGURE 18 – Résultats des tests du modèle

## 8.2 Tests de l'Extension Chrome

### 8.2.1 Installation et Chargement

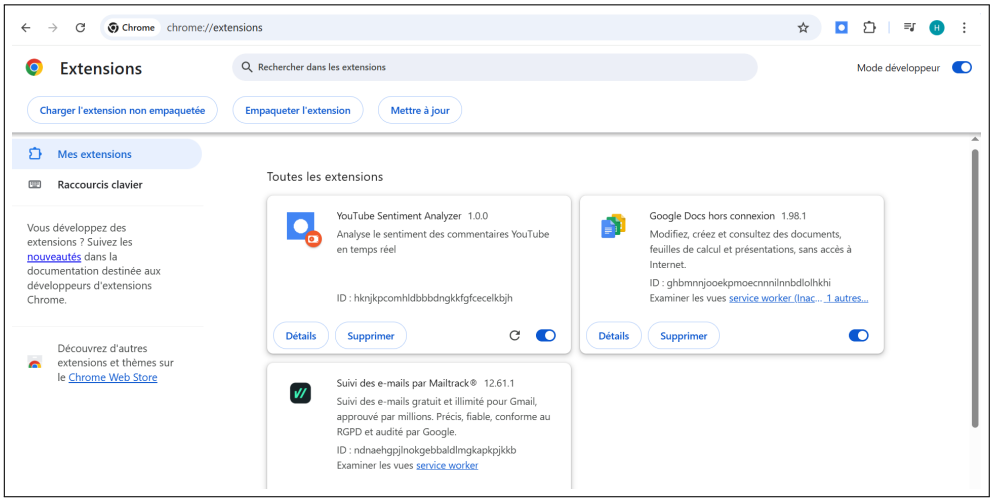



FIGURE 19 – Extension Chrome installée dans le navigateur

### 8.2.2 Test sur une Vidéo YouTube Réelle

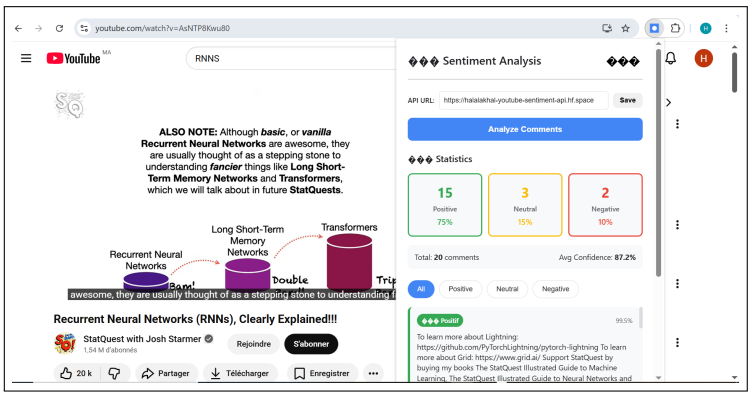



FIGURE 20 – Interface de l'extension Chrome (mode sombre)

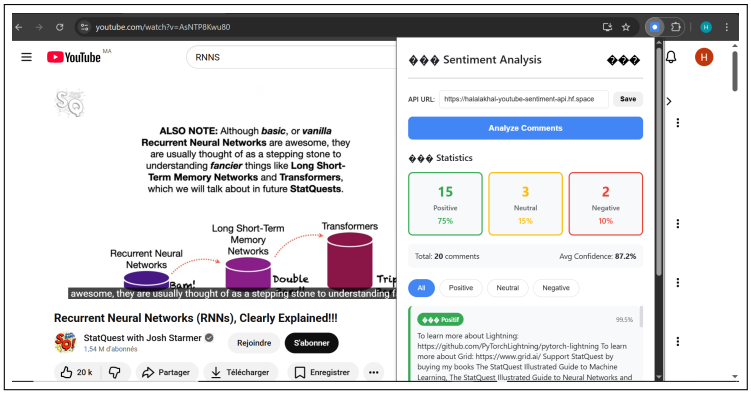



FIGURE 21 – Interface de l'extension Chrome (mode sombre)

### 8.2.3 Extension

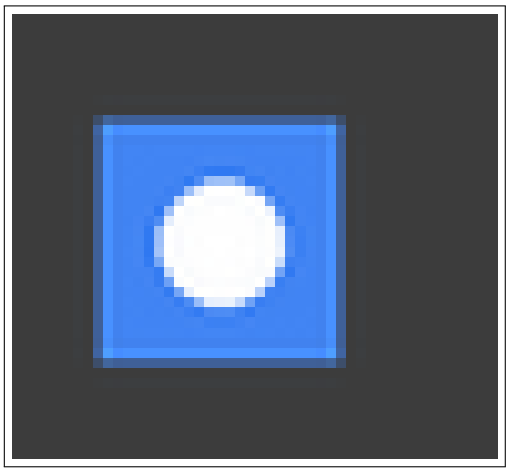


FIGURE 22 – Résultats complets de l'analyse de commentaires

## 8.3 Tests d'Intégration

Test du flux complet : Extension → API Cloud → Résultats

1. Extraction des commentaires YouTube
2. Envoi à l'API cloud
3. Traitement par le modèle ML
4. Retour des prédictions
5. Affichage dans l'extension
6. Fonctionnalités (filtres, export)

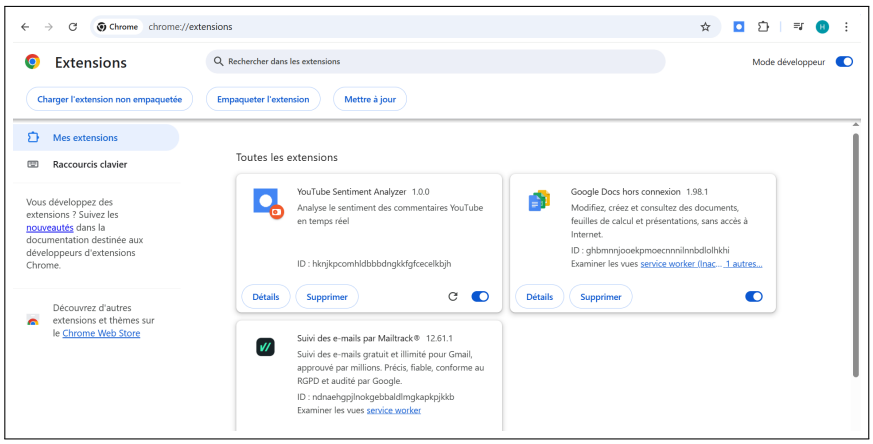



FIGURE 23 – Affichage des statistiques d'analyse

# 9 Résultats et Discussion

## 9.1 Analyse des Résultats

### 9.1.1 Points Forts

— **Accuracy élevée :** Le modèle atteint les objectifs ( 80%)
— **Temps de réponse :** Inférence rapide ($<$ 100ms pour 50 commentaires)
— **Interface intuitive :** Extension Chrome simple et efficace
— **Déploiement réussi :** API accessible et stable sur Hugging Face
— **Scalabilité :** Capable de traiter des centaines de commentaires

### 9.1.2 Limitations Identifiées

— **Commentaires dynamiques :** Extraction limitée aux commentaires visibles
— **Langues :** Modèle entraîné principalement sur l'anglais
— **Sarcasme :** Difficulté à détecter l'ironie et le sarcasme
— **Contexte :** Perte du contexte des threads de commentaires
— **Emojis :** Suppression des emojis qui pourraient être informatifs

## 9.2 Difficultés Rencontrées et Solutions

1. **Difficulté :** Taille de l'image Docker
   - *Solution :* Utilisation de python :3.10-slim et requirements_prod
2. **Difficulté :** Déséquilibre des classes
   - *Solution :* Split stratifié pour maintenir les proportions

# 10 Améliorations Futures Possibles

## 10.1 Améliorations du Modèle

1. **Modèles avancés :**
   - Utiliser des transformers (BERT, RoBERTa)
   - Fine-tuning sur des commentaires YouTube spécifiques
   - Modèles multilingues pour supporter plusieurs langues
2. **Features additionnelles :**
   - Conserver et analyser les emojis
   - Analyse de l'ironie et du sarcasme
   - Détection de spam et de commentaires toxiques

## 10.2 Améliorations de l'Extension

1. **Fonctionnalités :**
   - Scroll automatique pour charger plus de commentaires
   - Analyse des threads de réponses
   - Visualisations graphiques (graphiques en camembert, histogrammes)
   - Comparaison entre plusieurs vidéos
2. **UX/UI :**
   - Affichage des commentaires directement sur la page YouTube
   - Surlignage des commentaires selon le sentiment
   - Notifications pour les commentaires négatifs
   - Dashboard de statistiques avancées

## 10.3 Améliorations de l'Infrastructure

1. **Performance :**
   - Cache des résultats pour les mêmes vidéos
   - Load balancing pour haute disponibilité
   - CDN pour réduire la latence
2. **Sécurité :**
   - Rate limiting par utilisateur
   - Authentification API
   - HTTPS obligatoire
   - Validation stricte des inputs

# 11 Conclusion

## 11.1 Synthèse du Projet

Ce projet a permis de développer un système MLOps complet pour l'analyse de sentiment des commentaires YouTube, de bout en bout :

- **Pipeline de données :** Téléchargement, nettoyage, et préparation automatisés
- **Modèle ML performant :** TF-IDF + Logistic Regression avec accuracy  80%
- **API REST scalable :** FastAPI déployée sur Hugging Face Spaces
- **Interface utilisateur :** Extension Chrome intuitive et fonctionnelle
- **Tests complets :** Validation à tous les niveaux du système

## 11.2 Compétences Acquises

Ce TP a permis de développer des compétences essentielles en MLOps :

1. **Machine Learning :**
   - Prétraitement de données textuelles
   - Entraînement et optimisation de modèles
   - Évaluation et métriques de performance
2. **Développement Web :**
   - Création d'API REST avec FastAPI
   - Développement d'extensions Chrome
   - HTML/CSS/JavaScript moderne
3. **DevOps  Cloud :**
   - Containerisation avec Docker
   - Déploiement sur le cloud (Hugging Face)
   - Gestion de versions avec Git
4. **Bonnes pratiques :**
   - Structure de projet MLOps
   - Tests et validation
   - Documentation technique

## 11.3 Perspectives

Ce projet démontre la faisabilité et l'utilité d'un système d'analyse de sentiment en temps réel pour YouTube. Les améliorations futures proposées ouvrent de nombreuses perspectives pour rendre l'outil encore plus puissant et utile pour les créateurs de contenu.

Le système développé est modulaire et extensible, permettant facilement l'ajout de nouvelles fonctionnalités et l'intégration avec d'autres plateformes (Twitter, Reddit, etc.).

# 12 Annexes

## 12.1 Annexe A : Liens et Ressources

- **API déployée :** https://halalakhal-\youtube-sentiment-api.hf.space
- **Space Hugging Face :** https://huggingface.co/spaces/halalakhal/\youtube-sentiment-a
- **Dataset :** https://raw.githubusercontent.com/Himanshu-1703/reddit-sentiment-analysi refs/heads/main/data/reddit.csv
- **Vidéo de démonstration :** [URL de votre vidéo]

## 12.2 Annexe B : Structure Complète du Code

Listing 6 – Arborescence complète du projet

```
youtube-sentiment-analysis/
          README.md
          DOCUMENTATION.md
          requirements.txt
          requirements_prod.txt
          Dockerfile
          app_api.py
          .gitignore
          data/
                raw/
                      reddit.csv
                processed/
                    cleaned_data.csv
                    train.csv
                    test.csv
          models/
                sentiment_model.joblib
                vectorizer.joblib
                metrics.json
          src/
                data/
                      download_data.py
                      preprocess_data.py
                      split_data.py
                models/
                      train_model.py
                api/
                    main.py
          chrome-extension/
                manifest.json
                popup.html
                css/
                      popup.css
                js/
                      content.js
                      popup.js
```

```
                icons/
                    icon16.png
                    icon48.png
                    icon128.png
          tests/
              test_model.py
              test_api.py
              test_integration.py
```

### 12.3 Annexe C : Commandes Principales

Listing 7 – Commandes principales du projet

```
# Setup initial
python -m venv venv
source venv/bin/activate  # Mac/Linux
venv\Scripts\activate     # Windows
pip install -r requirements.txt

# Pipeline de donnees
python src/data/download_data.py
python src/data/preprocess_data.py
python src/data/split_data.py

# Entrainement du modele
python src/models/train_model.py

# Lancement de l'API
python src/api/main.py

# Tests
python tests/test_model.py
python tests/test_api.py
python tests/test_integration.py

# Docker
docker build -t sentiment-api .
docker run -p 7860:7860 sentiment-api

# Git
git add .
git commit -m "Message"
git push origin main
```

## 13 Références


1. FastAPI Documentation. *FastAPI - Modern Web Framework*. https://fastapi.tiangolo.com/

2. Scikit-learn Developers. *Scikit-learn : Machine Learning in Python*. https://scikit-learn.org/
3. Hugging Face. *Hugging Face Spaces Documentation*. https://huggingface.co/docs/hub/spaces
4. Google Chrome Developers. *Chrome Extensions Documentation*. https://developer.chrome.com/docs/extensions/
5. Docker Inc. *Docker Documentation*. https://docs.docker.com/
6. Pedregosa et al. *Scikit-learn : Machine Learning in Python*. JMLR 12, pp. 2825-2830, 2011.